

4.0

+0.4

4.4

Implementación (1.2/1.5)

- Diagrama de clases 11/11 (0.075/0.075)
- Atributos clase inicial 4/4 (0.075/0.075)
- Clase excepciones 3/6 (0.075/0.15)
- Implementación métodos (0.93/1.2)
 - Doc (0.2/0.2)
 - Métodos (0.73/1)
 - Bien 6
 - Regular 4 → 2
 - Mal 1 → 0

Buen trabajo!

Diseñando (0.88/1.25)

- Usuario (0.313/0.625)
 - Compras denegadas ✓
 - Post no confiable ✓
 - Post (0.563/0.625)
 - Promedio ✓
 - Activo sin stock ✓
 - Reseña por el mismo usuario ✓
- } No maneja excepciones

Extendiendo (0.95/1.25)

- Físicos: Peso ✓ Costo de envío ✓ Cobertura X 1/1 2/3
- Perecederos X Expiración ✓ Refrigeración Min Refrigeración X
 - Mejor diseño modelarlo como subclase de Físico, si no, se van a tener instancias (que no son perecederos) con esos atributos vacíos lo que representa gasto de memoria innecesario 0/1 2/3
 - Se sobre escribe canBePurchased ✓ 1/1 3/3
- Digitales ✓ Stock ✓ URL ✓ Tamaño ✓ 1/1
- Product Abstracto ✓

Conceptos (1/1)

Fecha	30	30	25	2:30	Institución	Escuela Colombiana de Ingeniería Julio Garzón
Alumno	Juan Daniel Bogotá Fuentes				Código	Materia
Curso	1	Semestre		Salón	1	2
Profesor	Angie Toboara Noreña @il				CALIFICACIÓN	

I. Implementación:

clase

- MercadoECS
- User
- Cart
- PurchaseDetail
- Product
- EciWallet
- Notification

métodos

1. connectCart
2. loadUser
3. connectCart
4. loadWallet
5. addNotification
6. isValid y getTotalPrice ✓
7. isValidPurchaseDetail ✓
8. canBePurchased ✓
9. canPaidCart ✓
10. - () ✓

Regiones de clases:

MercadoECS

- + connectCart (userId: int, walletId: String): boolean
- loadUser (userId: int): User

User

- + connectCart (cartId: String): boolean
- loadWallet (walletId: String): EciWallet

EciWallet

- + addNotification (n: Notification): void

Cart

- + isValid(): boolean
- + getTotalPrice(): double

PurchaseDetail

- + isValidPurchaseDetail(): boolean

Product

- + canBePurchased (productQuantity: int): boolean

EciWallet

- + canPaidCart (cart: Cart): boolean

Notification

- + - (message: String): Notification

clase responsable inicial:

```
public class MercadoECS {
    private ArrayList<Purchase> purchases;
    private HashMap<String, Product> products;
    private HashMap<String, Post> posts;
    private HashMap<Integer, User> users;
}
```

clase responsable de excepciones:

```
public class MercadoECSException extends Exception {
    public static final String USER_NOT_FOUND = "El usuario no existe en el sistema!";
    public static final String NO_CART = "El usuario no posee un carrito de compra que pueda ser validado!";
    public static final String INVALID_CART = "El carrito contiene al menos un producto con una cantidad superior a la disponible!";
}
```

Constructor de la clase MercadoECSException

```
public MercadoECSException (String Message) {
    super (Message);
}
```


Implementación del diseño:

on Notification:

/**

* constructor de la clase Notification

*/

public Notification(String message) { 1/1

this.message = ~~getMessage()~~;

actionDateTime = LocalDateTime.now();

}

on Cart:

/**

* método que obtiene el precio total del carrito de compra

* @return total que hay que pagar

*/

public double getTotalPrice() { 1/1

return totalPrice;

}

/**

* Método que valida si el carrito es válido

* @return true si es válido, false si no.

*/

public boolean isValid() { 0.5/1

for (PurchaseDetail pd : subOrderDetails) {

if (pd.isValidPurchaseDetail()) {

~~return true;~~

else {

return false;

}

}

return true;

on EciWallet:

/**

* Método que confirma si se puede pagar el carrito de compra

* @param cart el carro que se quiere pagar.

* @return true si se puede pagar, false si no.

* @throws MercadoException ~~si se ocurre algún error en el procedimiento.~~

*/

public boolean canPayCart (Cart cart) throws MercadoException { 1/1

double total = cart.getTotalPrice();

if (balance < total) {

throw new MercadoException (MercadoException.INSUFFICIENT_BALANCE);

} else {

return true;

} else {

return false;

}

on Product:

/**

* Método que confirma si puede ser pagado el producto

* @param productQuantity la cantidad de stock del producto

* @return true si se puede pagar, false si no.

*/

public boolean canBePurchased (int ProductQuantity) { 0/1

if (ProductQuantity > 0) {

return true;

} else {

return false;

}

return productQuantity <= stock;

en PurchaseDetail:

- * Método que verifica los detalles de la compra con validos
- * @return true si son validos, false si no

```
//  
public boolean isValidPurchaseDetail() {  
    if (product.canBePurchased()) {  
        return true;  
    } else {  
        return false;  
    }  
}
```

1/1

en User:

- * Método para agregar una notificación a la lista de notificaciones

```
//  
public void addNotification(Notification n) {  
    notifications.add(n);  
}
```

1/1

//

- * Método para cargar la billetera por su Id
- * @param walletId para cargar la billetera por su respectivo Id
- * @return wallet. retorna la billetera

* @throws

```
private EciWallet loadWallet(String walletId) throws MercadoEciException {  
    for (Wallet w : EciWallet) {  
        if (w.getId().equals(walletId)) {  
            return wallet;  
        } else {  
            throw MercadoEciException (MercadoEciException.No_WALLET_FOUND);  
        }  
    }  
}
```

throws MercadoEciException 0.5/1

Se tiene que mirar si ninguna wallet es la que tiene el id y ahí si lanzar la excepción

//

- * Método para confirmar si se puede pagar el carrito de compra con la billetera
- * @param walletId billetera con la que se quiere pagar el carrito
- * @return true si se puede, false si no

```
//  
public boolean canCheckOutCart(String walletId) throws MercadoEciException {  
    if (cart == null) {  
        throw MercadoEciException (MercadoEciException.NO_CART);  
    }  
    if (cart.isValid()) {  
        throw MercadoEciException (MercadoEciException.INVALID_CART);  
    }  
}
```

0.5/1

try { boolean canPay = false

wallet wallet = loadWallet(walletId);

← Afuera del try porque no se atiende esa excepción

boolean canPay = w.canPayCart(cart);

} catch (Exception e) {

m = e.getMessage();

}

if (!canPay) {

Notification n = new Notification(m);

addNotification(n);

} else {

return true

}

}

Se podía hacer dentro del catch si se lanzaron las excepciones no se pudo pagar

II. Métodos:

- Método para cargar el usuario por su id
- @param userID el id del usuario que se quiere cargar
- @return usuario
- @throws NoSuchElementException si ocurre algún error

```
public User loadUser(int userID) {
    for (User u : users) {
        if (u.getId() == userID) {
            return u;
        }
    }
    throw NoSuchElementException("Usuario no encontrado");
}
```

0.5/1

Se lanza la excepción cuando ningún usuario es el del id

- Método que verifica si el carrito de compra de un usuario puede ser pagado
- utilizando una wallet específica.
- @param userID el usuario que pagará el carrito.
- @param walletID la wallet del usuario que pagará el carrito.
- @return true si se puede pagar, false si no.
- @throws NoSuchElementException si ocurre algún error.

```
public boolean canCheckout(int userID, String walletID) throws NoSuchElementException {
    User u = loadUser(userID);
    if (u.canCheckoutCart(walletID)) {
        return true;
    }
    return false;
}
```

return u.canCheckoutCart(walletID)

III. Excepciones:

- La clase abstracta o mejor cuando hay clases que comparten atributos, por ejemplo la clase figura o mejor tratarla como abstracta ya que puede compartir muchos cosas con sus subclases, la interfaz o mejor cuando solo hablamos de métodos que ciertas clases que heredan que las implementan, por ejemplo un certificado. Aquí solo se puede extender una clase abstracta, y la interfaz la podemos implementar las veces que necesitamos.
- El mayor beneficio es tener los mensajes personalizados, por ejemplo, si ocurre un error y no está personalizado es muy difícil saber que hizo la excepción, más que es mucho más ordenado, y fácil para el usuario saber por qué saltó la excepción.

III. extendiendo:

1.

```

+ Producto
- atributo que va ahí
+ hasStock: boolean

```

```

+ FisicProduct
- peso: double
- contenido: double
- material: boolean
- fechaExpiracion: LocalDateTime
- fechaExpiracion: LocalDateTime

```

```

+ Digital Product
- url: String
- tamaño: double
- stock: boolean = true
+ hasStock: boolean

```

Se modelaba como una subclase de Producto Físico

Observaciones

```

+ puedeSerVendido: boolean
+ estáDisponible: boolean
+ hasStock: boolean

```


30	10	25	230	Examen	Calificación	Código	Materia	Dopo
Alumno				Juan Daniel Aguilar Torres		Código		Materia
Curso	1	Alimentación	Semestre	Salón	Hoja No	2	de	2
Profesor				Angie Talara Medina Gil		CALIFICACIÓN		

2.

public abstract class Product {

private String id;
private String name;
private String category;
private double price;
private String description;
private int stock;

public class FridgeProduct extends Product {

private double peso;
private double capacidad;
private boolean refrigerado;
private LocalDateTime fechaExpiración;
private LocalDateTime fechaRefrigeración;
private boolean disponible;

public class DigitalProduct extends Product {

private String url;
private double tamaño;
private stock digital boolean stock = true;

3.

los cambios que se deben realizar debería ser la validación del tipo de producto que se desea adquirir, así se sabe si el producto físico está disponible, básicamente solo sería en el método que valida el carrito de compra. puede que un producto físico lleve más de 5 días refrigerado, así el carrito debe ser inválido.

4.

Si se cumple ya que está abierto a extensión y cerrado para modificación, además se usa clase abstracta para hacer el código extensible, no se necesita modificar código para agregar más cosas.

II Diseñando:

